Applying AmberMeta to Deposited MD Data

Standard Operating Procedure

Michele Bonus • AmberMeta v1.1.0 • November 10, 2026

1 What this procedure is for

AmberMeta reads a directory of AMBER simulation files and reconstructs what was actually run: which topology belonged to which stage, which run continued which, how long each ran, and where the record is incomplete. The output is a *manifest*, a single YAML file describing the campaign, together with the summary artifacts of §4.5.

This procedure covers depositing one *system* (one directory under the collection root). Work through it once per system. It uses the graphical interface throughout; every step has a command-line equivalent if you need to script it across many systems (§6).

> **The one thing to understand first**
>
> AmberMeta reports only what your files support. It will not invent a temperature it cannot read or assert a continuation it cannot demonstrate. If a deposit is missing its output logs, AmberMeta will say so rather than guess. That is the point, but it does mean an incomplete deposit produces a thin manifest.

2 Before you start: what a complete deposit contains

AmberMeta expects the file kinds below, the topology once per system and the rest once per run. The middle column is what is lost if that kind is absent.

File	What only this file can tell AmberMeta	Typical suffix
Topology	Atom count, box, whether masses were repartitioned	`.prmtop`, `.top`
Input	Requested settings: steps, timestep, thermostat, barostat	`.mdin`, `.in`
Output log	What *actually* ran: achieved time, temperature, completion, timings	`.mdout`, `.out`
Trajectory	Frame count, frame spacing, box volume over time	`.nc`, `.mdcrd`
Restart	The coordinates handed to the next run, the link in the chain	`.restrt`, `.rst`

The output log is the one most often missing and the most costly to lose. A deposit holding trajectories and restarts but no `.mdout` files can be assembled into a manifest, but nothing in it can be confirmed: every run's duration is unverified and every junction between runs is reported as an unexplained gap. Keep the logs.

> **Check before you deposit**
>
> Confirm every directory is readable (`chmod u+rx` on directories, not just files) and that no editor crash files, core dumps, or scratch copies are sitting among the run files. AmberMeta skips what it does not recognise, but a stray file that *looks* like a run, such as a lone `.in` in an analysis folder, becomes a step you then have to remove by hand.

3 Recommended layout for new work

For simulations you are about to run, this layout lets AmberMeta group everything correctly with no manual intervention:

```
system/
  topology/     system.prmtop, system.inpcrd
  minimization/ 01_min.in, 01_min.out, 01_min.restrt
  equilibration/
    run1/       02_heat.in, 02_heat.out, . . .
    run2/
  production/
    run1/       prod_0001.in, prod_0001.out, prod_0001.nc, . . .
    run2/
```

Two rules carry all the weight:

1. **Replicas are sibling directories directly inside the stage they belong to**, named identically across stages (`run1`, `run2`, . . . or `01`, `02`, . . .). This is what lets AmberMeta recognise them automatically.
2. **The replica identifier appears in the directory name only, never in the filenames.** Use `prod/01/prod_0001.out`, not `prod/01/prod_01_0001.out`. Repeating the replica number inside the filename defeats automatic grouping (§5).

Chunked runs continue to use a numeric suffix (`prod_0001`, `prod_0002`); AmberMeta chains them by reading each run's restart, not by the numbering.

4 Procedure

4.1 Launch

Run AmberMeta at the top of the system directory you are depositing. That directory becomes a hard boundary: nothing outside it is read.

```
ambermeta gui /path/to/sys023
```

A browser opens on `http://127.0.0.1:8765`. The left pane lists the files it found, typed by what they are rather than by extension (Figure 1). Confirm the file counts look right before going further; if a whole stage is missing here, it is a permissions or path problem, not an AmberMeta one.

4.2 Discover

Press **Discover**. AmberMeta walks the tree, groups files into runs, infers each phase's role from the inputs, and links each run to the one whose restart it consumed. The result is a timeline (Figure 2).

Read down the timeline and check three things:

- **Phase roles.** Minimization, heating, equilibration and production should be labelled as such. A phase labelled *Unassigned* usually means a preparation step (tLEaP, ParmEd) was picked up as a run. Set its role or delete it.
- **The chain.** Between consecutive steps you should see *restart of* . . . . A step reading *starting structure* part-way through a production sequence means the link could not be established.
- **Topologies.** Where several topologies exist, check each phase is bound to the right one. A repartitioned-mass topology is tagged HMR.

4.3 Define replicas

If the system was run as replicas, tell AmberMeta so. Runs in different replicas are independent trajectories; without this they are treated as one sequence and the junctions between them are reported as gaps.

Press **Define replicas. . .** . If the layout follows §3, the members are already proposed, so accept them. Otherwise pick the directory level that distinguishes the replicas and accept only the members that really are replicas (preparation directories at the same level are offered too, and should be left alone).

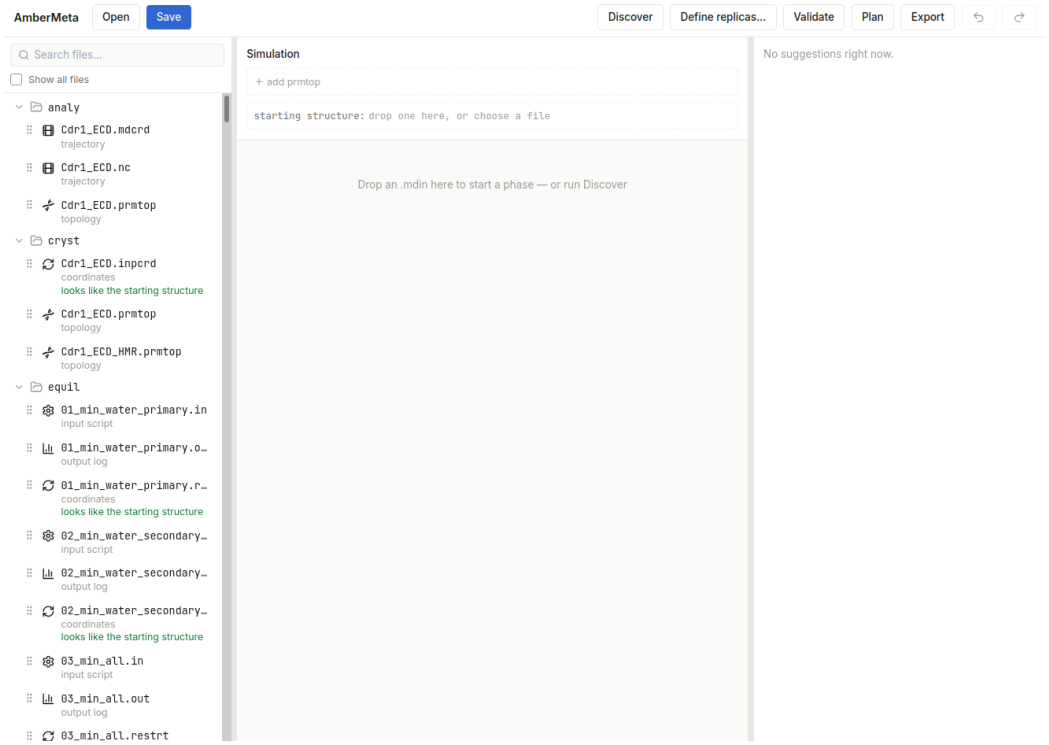


Figure 1: The file pane after launching against a system directory. Files are typed on sight (*topology*, *input script*, *output log*, *coordinates*), and candidates for the starting structure are flagged.

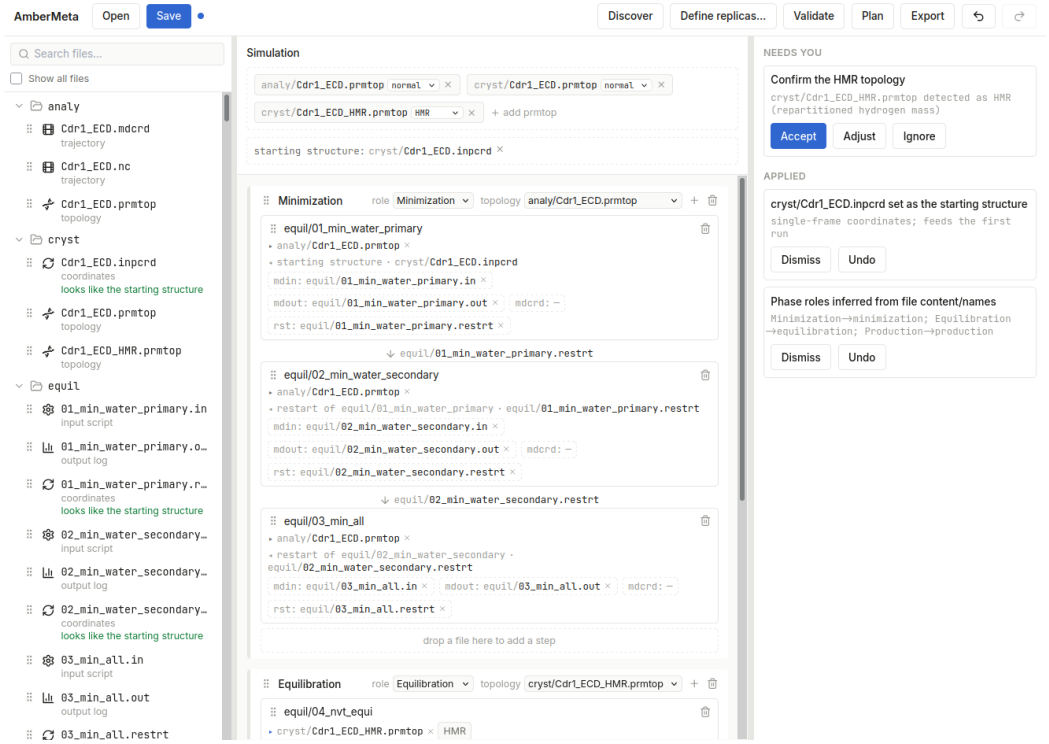


Figure 2: After **Discover**. The centre pane is the reconstructed timeline, with each run's inputs, outputs and the restart it continues from. The right pane separates what AmberMeta did on its own (APPLIED) from what it needs you to confirm (NEEDS YOU).

Once accepted, each phase carries a banner naming the member and its run count (Figure 3).

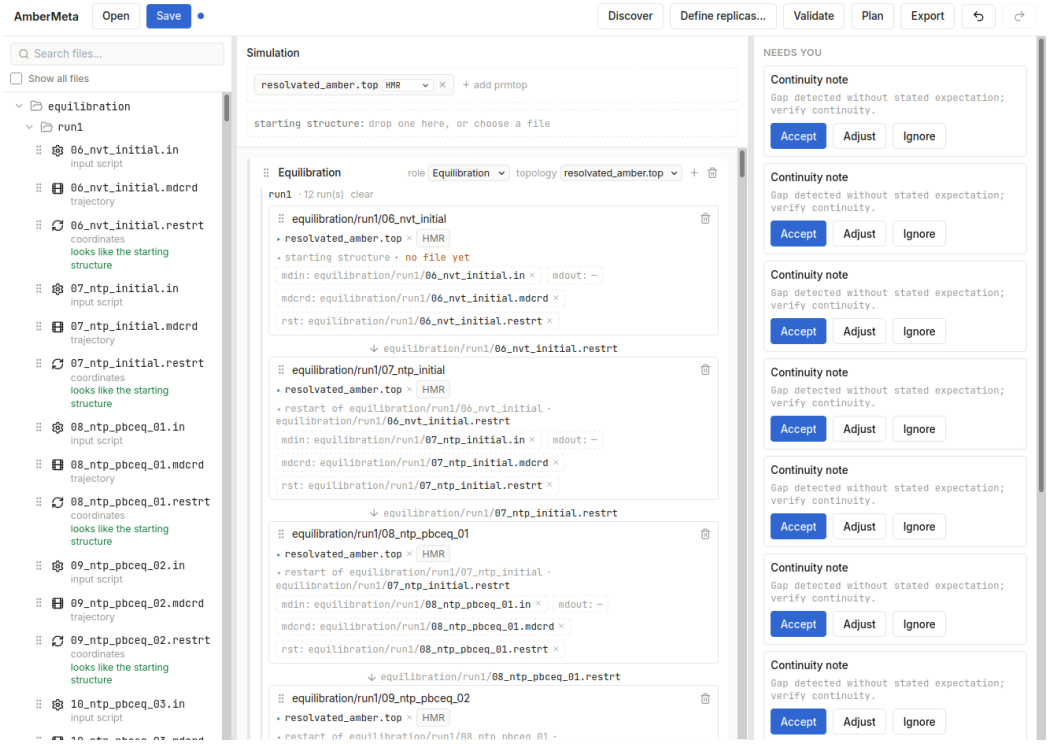


Figure 3: A system with replicas accepted: the `run1` banner above the timeline marks the member and its twelve runs. The continuity notes on the right are this deposit's missing output logs: with no `.mdout`, no junction can be confirmed.

4.4 Validate, and resolve what it raises

Press **Validate**. Findings arrive in the right-hand tray in two groups:

APPLIED
: Inferences AmberMeta already acted on. Skim these, because they are claims about your data. The starting structure and phase roles usually land here. To reverse one, change what it set in the timeline: clear the starting structure, or pick another role for the phase.

NEEDS YOU
: Questions it will not answer on its own. A card about one step has **Show step**, which opens that step in the inspector. **Dismiss** hides a card for the session and resolves nothing: fix the cause in the timeline, then press **Validate** again.

The findings you are most likely to see, and what they mean:

Finding	What to do
Confirm the HMR topology	Check the flagged topology really is the mass-repartitioned one. If it is not, set its kind with the selector next to it in the pool.
Gap detected without stated expectation	Two consecutive runs do not meet in time. Either a run is missing from the deposit, or the gap is real and deliberate. Say which.
Atom count mismatch across files	A step is bound to the wrong topology. Rebind it in the timeline.
Trajectory is truncated or still being written	The run is incomplete or was copied mid-write. Re-copy it, or accept that its times come from the log instead.

Resolve findings until the only ones left are those you can explain. Dismissing a card only tidies the tray: **Validate** and the summary still report the finding. A finding left unresolved is an unanswered question travelling with your data.

4.5 Save, then Plan

Press **Save** to write `manifest.yaml` into the system directory. That is the deposit artifact.

Then press **Plan**, and keep the files it writes. **This step is required, not optional.** Plan writes `summary.json` and `methods_summary.json` beside the manifest: the manifest records what your files *are*, and these record what AmberMeta *made of them*, run by run, with every number it derived.

That second record is what lets us audit the parser against real data. AmberMeta is being developed against this collection, and the failures worth finding are the ones that occur on layouts and AMBER versions nobody anticipated. Those are invisible in a manifest and obvious in a summary: a duration that disagrees with the trajectory, a run reported as unfinished that plainly finished, a continuity gap where the chain is sound. If a system's summary files are not deposited, its quirks stay undiscovered until someone hits them in an analysis.

So: deposit `manifest.yaml`, `summary.json` and `methods_summary.json` together, and report anything in them that contradicts what you know about the runs.

Re-running this procedure later re-reads the files; unchanged files are not re-parsed, so a second pass over an unchanged system is quick.

5 Applying this to existing deposits

Data already collected does not follow one convention. The four encountered so far, and what each needs:

Layout	Example	Action
Replicas inside each stage	`equilibration/run1..run6`	Accept the proposal.
Numbered replicas in a stage	`prod/01..05`	Accept the proposal.
Replicas beside the stages	`run1..run4` beside `equilibration/`	Pick the top level in **Define replicas. . .**, then accept only `run1..run4`.
Replica id also in filenames	`prod/01/GB1_204_01_000`	Pick the replica level manually, then accept only the numbered members.

In the last two cases the automatic proposal is declined rather than wrong: AmberMeta refuses to guess when a layout admits more than one reading (Figure 4). Choosing the level yourself is the intended route, and the resulting manifest is identical to one from a conforming layout.

6 Quick reference

Step	Command-line equivalent
Launch	`ambermeta gui DIR`
Discover	`ambermeta discover DIR`
Discover + save	`ambermeta discover DIR --write manifest.yaml`
Build & check	`ambermeta plan DIR --recursive`
Summaries	`ambermeta plan DIR --recursive --summary-path summary.json`
Check one file	`ambermeta validate FILE [FILE . . . ]`
Inspect one file	`ambermeta info FILE`

Note that `validate` checks individual files; the whole-campaign check the **Validate** button performs is what `plan` does while assembling the protocol.

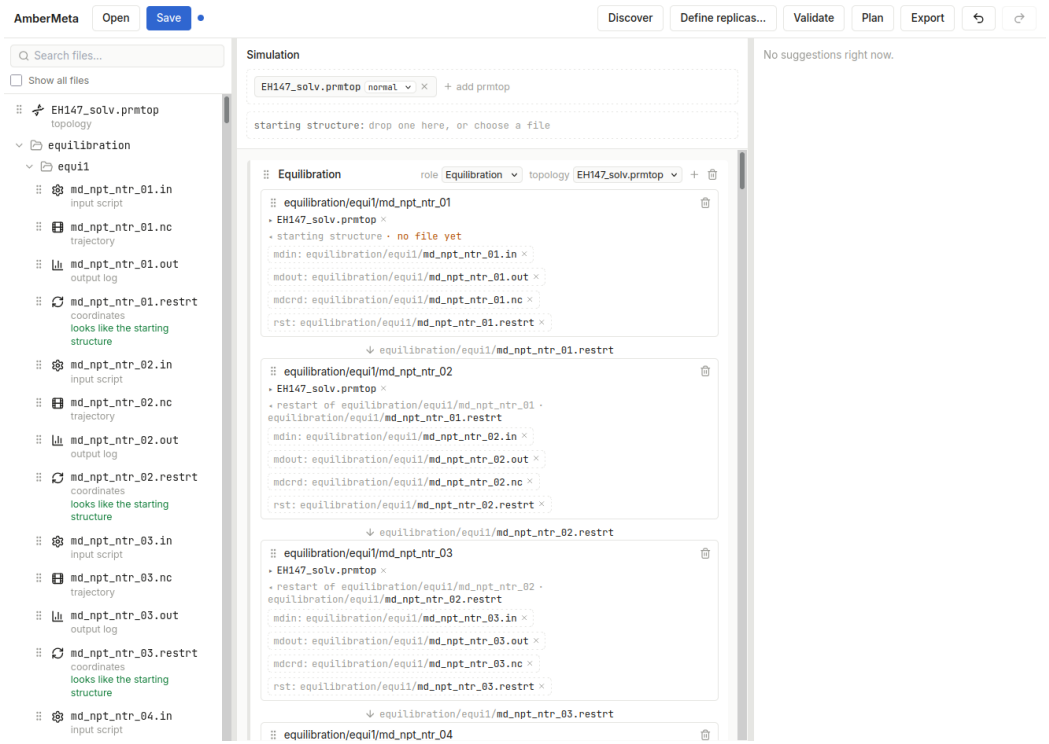


Figure 4: A layout where replicas were not proposed automatically: the runs are all present and correctly chained, but split across many phases with no member grouping. **Define replicas. . .** is what resolves this.

Checklist per system

1. Every directory readable; no stray files among the runs.
2. Output logs present for every run.
3. Discover run; phase roles and the restart chain read correctly.
4. Replicas defined, if the system has them.
5. Validate run; every finding accepted, adjusted or deliberately ignored.
6. `manifest.yaml` saved into the system directory.
7. **Plan** run, and `summary.json` and `methods_summary.json` deposited with it.